Report of Real-time Face Recognizer

https://github.com/qinglin0624/CV-Pytorch
Xuchen Gong

I Face recognition with FaceNet

1.1 Data pre-processing and loading

Using the MTCNN trained in the last week, all the images in IFW dataset are aligned, cropped and then saved in a folder. After a face is detected, the face will be cropped from the image by the box that is 40% enlarged. When no box or several boxes are detected, the original image will be saved, and there are totally 1335 out of 13233 images saved originally.

Figure 1. Aligned and cropped faces.

For data loading, all the people with more than one image available will be a potential anchor, and this customized dataset has 1184 names of potential anchor for training and 496 for evaluation. In every epoch, a name is picked in order and two images of that person will be selected randomly, one as *anchor* and the other as *positive*. Then an image of another person will be selected randomly from the whole dataset and used as *negative*. Moreover, as an augmentation, each image has a probability of 0.5 to be horizontally flipped whenever it is loaded.

1.2 Models

To compare the effects of embedding, I tried two models, FaceNet and ResNet18, for the face recognition task. ResNet18 has a 128D classification layer as the last layer and FaceNet has a 128D embedding instead, which is obtained by adding L2 norm and a multiplication of 10 after the classification layer.

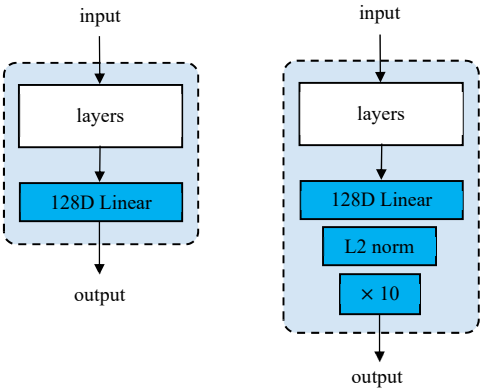

Figure 2. Architecture of two models. **Left**: ResNet18. **Right**: FaceNet adopted from ResNet18.

1.3 FaceNet

As mentioned in paper, the margin is selected to be 0.5 and the triplet loss is only calculated on hard triplets.

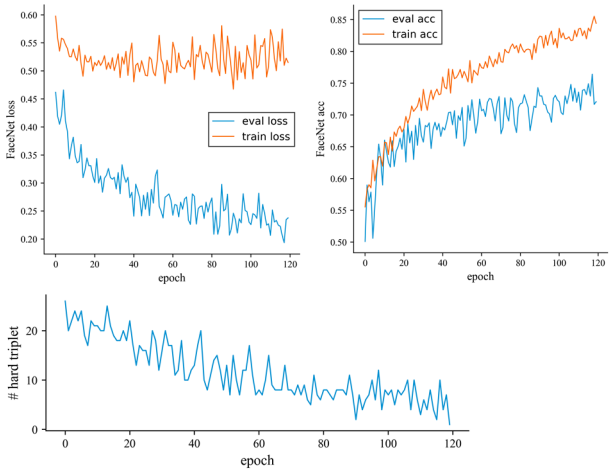

Figure 3. Train FaceNet. **Top Left**: Loss. **Top Right**: Accuracy. **Bottom**: Number of hard triplets in the last batch.

1.4 ResNet18

For ResNet18, the margin is selected to be 0.5 as well and two trials are carried out. One has the triplet loss calculated on hard triplets only (denoted as "hard" mode) and the other has the triplet loss calculated on all the triplets (denoted as "easy" mode).

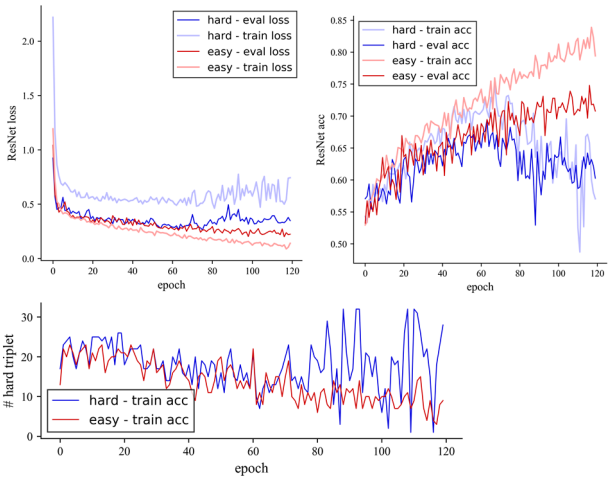

Figure 4. Train ResNet18. **Top Left**: Loss. **Top Right**: Accuracy. **Bottom**: Number of hard triplets (in the last batch) in the last batch.

As seen in Fig. 4 (Top right), when the triplet loss is only calculated on hard triplets (the "hard" mode), after around 60 epochs the loss begins to oscillate, accuracy displays a tendency to decrease, and # hard triplet begins to vibrate drastically. One possible explanation is that after 60 epochs, the model has learned the good images well and the remaining hard triplets are of poor quality. Therefore, poor training begins to occur because poorly imaged faces dominate the hard triplets. However, the nice accuracy curve of FaceNet trained in the hard mode raises

another question. According to Fig. 3 (Bottom), when training FaceNet in the hard mode, # hard triplet decreases stably below 5 and accuracy continuously increases, which shows that in my case, poor training does not necessarily come together with the hard mode. As far as I know, L2 normalization constrains the representation on a hypersphere and avoids scaling the embedding correspondingly when the margin is scaled. However, whether it is L2 norm that solves the problem of poor training in the hard mode and why it is the case is to be found.

1.5 Comparison

In this section the test results of FaceNet and ResNet18-easy are plotted. They have similar performance and still have a lot of room for improvement, since as shown in Fig. 5 (left), the distances they output are still relatively indiscriminating.

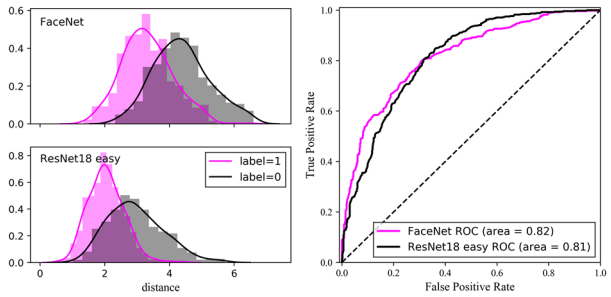

Figure 5. Compare test results of FaceNet and ResNet18-easy. **Left**: Calculated distances of the tested pairs. **Right**: ROC curve of two models.

1.6 Errors

Tested on the parisDevTest.txt provided by IFW dataset, FaceNet has an accuracy of 0.735. It makes 255 mistakes in total, out of which 60% are *false accept* and 40% are *false reject*.

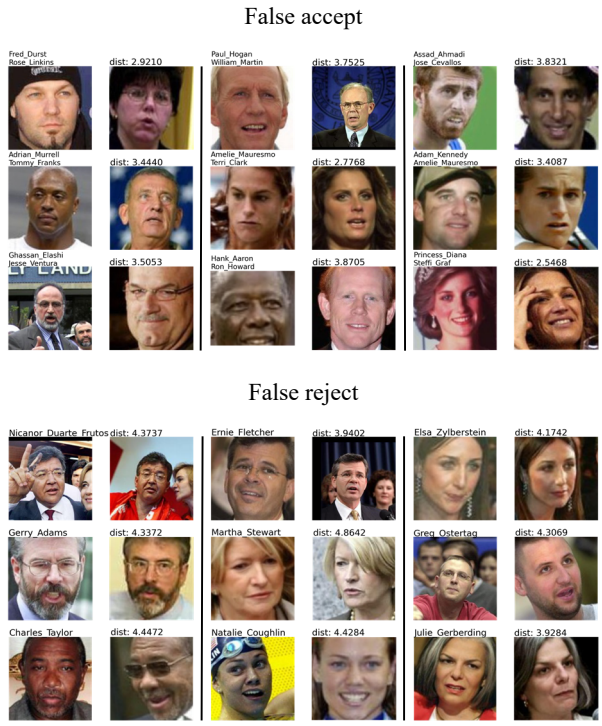

Figure 6. Some randomly selected *false accept* and *false reject* errors made by FaceNet.

Comment

Two of the reasons for *false reject* are summarized below.

Facial color and brightness & contrast variance

As seen in the case of Gerry Adams and Charles Taylor above, the facial color of a person varies in different occasions. As seen in the case of Elsa Zylberstein, the performance of the model is also vulnerable to variances in the brightness and contrast of images.

Poorly aligned and cropped

As seen in the case of Amie Fletcher, Greg Ostertag and Tim Allen, the poorly cropped images add more noise and may lead to failures. Since there are around 10% of the images saved originally because of failure in detection, the quality of the dataset is largely damaged. Therefore, a better response to detection failure is to use LFW alignment instead, as suggested in the FaceNet paper.

II Real-time face recognizer

2.1 Real-time face recognition (MTCNN + FaceNet)

Because of the unavailability of GPU on local PC, this real-time project is modified to allow for real-time processing while later visualization. During the real-time processing, all the information needed for visualization, including the predicted bounding boxes, landmarks, and names, is stored in a dictionary which contains three corresponding keys. The value of each key is a list whose length is equal to the number of frames of the video. After the processing is finished, the stored dictionary is loaded and used for visualization carried out on the local PC.

2.2 Method

Generally, the progress of this project is "MTCNN → Face alignment → FaceNet". Specifically, the progress can be divided into "Candidate preparation", "MTCNN phase" and "FaceNet phase".

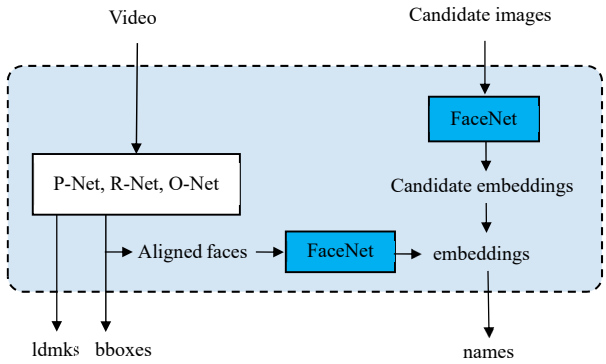

Figure 1. Workflow of real-time processing.

Candidate preparation

The candidates are limited to the characters in the video. Each candidate has an image cropped from the video and its embedding is calculated and then stored in a dictionary.

MTCNN phase

In real-time processing, each frame of the video is independently fed into MTCNN in order, and the outputs include the landmarks and the bounding boxes. The bounding boxes are then used for

face alignment, the result of which is then fed into FaceNet.

FaceNet phase
The FaceNet calculates the embedding of the given face, which is then compared with the candidate embeddings. If no candidate embedding is close enough to the output, the face is labeled "unknown". Otherwise, the face will be named after the name corresponding to the embedding closest to the output.

In addition, as introduced by a GitHub repository (https://github.com/ageitgey/face_recognition), two techniques that fasten the processing are implemented. Firstly, process each video frame at 1/4 resolution (though still display it at full resolution during visualization). Secondly, only detect faces in every other frame of video. Since in my case the video chosen for demo is short, the second technique is not necessarily used.

2.3 Demo

The video used for demo is a cut of the movie *The Shawshank Redemption* where six American men appear in the frames. The diversity of appearance among the six men includes age (middle-aged vs old) and complexion (the blacks vs the whites), as shown in Fig. 2.

Figure 2. People in demo.

This video imposes two difficulties. Firstly, the brightness as well as the contrast of the faces is low, which makes the complexion and other features harder to be differentiated. Secondly, they are of the same gender and nationality, and two of the six, Man1 and Man2, have relatively illegible appearance.

It is expected that Andy, Red, Brooks and Heywood are easy to tell while Man1 and Man2 are relatively harder to be differentiated, and the result generally agrees with this expectation.

Figure 3. Screenshots of demo.

In terms of bad performance, there are generally three types.

Unstable box size
The variation of the size of boxes leads to unsmooth transition between frames.

False recognition
As shown in the demo for MTCNN two weeks ago, the models tend to mistake letters and numbers for faces.

Diffident naming
The clustering effect of FaceNet is not impressive and different people are not differentiated well by their embeddings. Therefore, this project returns names in an unstable manner.

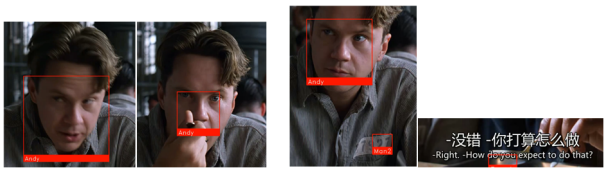

Figure 4. Bad performance. **Left**: Unstable box size. **Right**: Mistake letters and numbers for faces.